

OctaC: A Language Specification

Designing a Hybrid Language for Numerical and Matrix Computation

Syed Taha

Muhammad Usman

October 9, 2026

Contents

Foreword and acknowledgments	5
Preface	7
1 Introduction	9
1.1 Motivation	9
1.2 Design Influences	9
1.3 Features	10
1.4 Sample Programs	10
1.4.1 Sum of Squares	11
1.4.2 Matrix Multiplication	11
1.4.3 Matrix-Vector Product	12
1.4.4 Iterative Convergence	12
1.4.5 Input Validation	13
1.4.6 GPU-Accelerated Matrix Operations	13
1.5 Looking Ahead	14
2 Lexical Analysis and the Lexer	15
2.1 Token Specification	15
2.2 Deterministic Finite Automata	15
2.2.1 Character Classes	17
2.2.2 States	17
2.2.3 Identifiers, Keywords, Type Names, and Bool Literals	18
2.2.4 Punctuation	19
2.2.5 Operators	20
2.2.6 Constants	21
2.2.7 Compiler Tokens	22
2.2.8 Skipped Input	23
2.2.9 The Merged DFA	23
2.3 The Lexical Analyzer Algorithm	25
2.3.1 Maximal Munch	25
2.3.2 The Scanning Loop	25
2.3.3 Resolving the Token	25

2.3.4	Errors	25
2.3.5	Token Output	27
2.3.6	Sample Runs	27
2.4	Lexers in the Real World	29
2.4.1	Lexer Generators	29
2.4.2	Hand-Written Lexers	30
2.4.3	What Real Lexers Handle	30
2.4.4	Simplifications in OctaC	30
3	Syntax Analysis and the Parser	33
3.1	Grammar Theory	33
3.1.1	Left-Recursive Grammars	33
3.1.2	LL(k) Grammars	34
3.2	The Grammar	34
3.2.1	Notation	34
3.2.2	Program and Functions	35
3.2.3	Types	35
3.2.4	Statements	35
3.2.5	Expressions	36
3.3	How the Parser Works	38
3.3.1	One Function per Nonterminal	38
3.3.2	Matching Terminals	39
3.3.3	Building the Tree	39
3.4	Errors and Recovery	39
3.4.1	Recovery	39
3.5	Sample Parses	40
3.5.1	A Minimal Program	40
3.5.2	Operator Precedence	42
3.6	Parsers in the Real World	43
3.6.1	Parser Generators and Parsing Methods	43
3.6.2	Hand-Written Recursive Descent	44
3.6.3	Error Recovery	44
3.6.4	Simplifications in OctaC	44
	Conclusion	45
	Bibliography	48

Foreword and acknowledgments

This text details the design of OctaC, a statically typed, C-structured language for numerical and matrix computation, along with the compiler built to implement it. The chapters assembled here document the language specification and the design decisions made along the way.

We acknowledge the support and guidance of our supervisor, Miss Sadaf Alvi, whose feedback helped shape the project goals and technical approach.

The work was carried out by Syed Taha and Muhammad Usman. It was completed as part of the Compiler Construction course at our university. For questions, bug reports, or further information about the implementation, please contact Syed Taha at s.taha.29208@khi.iba.edu.pk.

Preface

OctaC is a statically typed, C-structured language for numerical and matrix computation. Dynamically typed languages like Octave [2] let a programmer multiply two matrices of incompatible shapes without complaint until the operation actually runs. A statically typed language can catch that mistake before a single instruction executes, but only if the compiler is taught to treat shape, not just type, as a property of a value that can be checked at compile time. OctaC brings that discipline to a C-structured surface syntax, with scalar types (`i16`, `i32`, `i64`, `f16`, `f32`, `f64`, `bool`) alongside matrix and vector types parameterized by their element type (`matrix<i32>`, `vector<f32>`).

This is a living book. It begins with an introduction to OctaC and its lexical analysis, and is meant to grow chapter by chapter as the compiler is built over time: the formal grammar, the parser, the semantic analysis pass, and code generation will each be added as their own chapter once they exist. Chapter 1 introduces the language as it currently stands, and Chapter 2 covers its lexer.

This book prioritizes clarity and feasibility over completeness. Where we simplify (omit a feature, restrict a rule, choose one semantic over another), we try to say so explicitly rather than let the omission speak for itself.

We hope it serves as both a reference for the finished language and an honest record of the decisions, and trade-offs, behind it.

```
function main() -> i32 {  
    printf("Hello, world!\n");  
    return 0;  
}
```


Chapter 1

Introduction

OctaC is a small, statically typed language for numerical and matrix computation. Its purpose is to make numerical scripts, matrix operations, and iterative computations easy to read and easy to write, without giving up static typing.

This chapter introduces the language at a high level: the motivation behind it, the languages it draws on, the features it offers, and the shape of its syntax through a set of sample programs. The precise definition of its tokens, and the lexer that recognizes them, follows in Chapter 2.

1.1 Motivation

Languages built for matrix work, such as Octave and MATLAB, make matrices and vectors easy to write, but they are dynamically typed [2, 13]. A program that multiplies two matrices of incompatible shapes is accepted as written and fails only when the operation runs. Languages such as C and C++ check types before a program runs, but treat matrices as library structures or nested arrays, and leave the arithmetic to hand written loops [6, 10].

OctaC sits between the two. Matrices and vectors are built in types, and their arithmetic is part of the language. This requires the semantic analysis phase to track shape alongside type: operations on statically known shapes are checked at compile time, while operations on shapes that depend on a function parameter, a return value, or a loop built matrix are checked at runtime.

1.2 Design Influences

OctaC is a hybrid rather than a subset of a single language. Each part of its design comes from the language that handles it best:

- **Octave and MATLAB** for matrix and vector arithmetic [2, 13].
- **C and C++** for block structure and statement syntax [6, 10].
- **Rust and Zig** for explicit sized primitive types [7, 17].

- **Python** for range based and `for...in` iteration, and for English word logical operators (`and`, `or`, and so on) [9].

1.3 Features

Native matrix and vector types. Matrices and vectors are built in types, not library structures. Operators are split between true matrix arithmetic (`*` for multiplication, `'` for transpose) and element wise arithmetic (`.*`, `./`, and related operators), so the operation being performed is visible at the point of use.

Beyond this, OctaC supports:

- Explicit sized variable declarations (`i32`, `f32`, `bool`, `matrix<i32>`, `matrix<f32>`, and related types)
- Block scoping with shadowing in nested blocks
- Scalar, matrix, and element wise operators
- Assignment, including compound forms (`+=`, `*=`)
- Conditionals: `if` / `else if` / `else`, using English word logic (`and`, `or`, `not`, `is`)
- Loops: range based `for`, `for...in`, `until`, `do...until`
- Functions with typed parameters and an explicit return type
- Matrix and vector literals and indexing

1.4 Sample Programs

The following programs show the syntax of OctaC in use. Each one is a complete program, followed by a walkthrough of the constructs it introduces. Later programs build on what earlier ones explain, so they are best read in order.

A few things are common to all of them. A program is made of functions, and execution starts in the function named `main`, which returns an `i32`. Statements end with a semicolon, and blocks are delimited by braces, as in C. A comment starts with `//` and runs to the end of the line. Output is written with `printf`, whose format string follows the one in C [6]: `%d` prints an integer and `%.2f` prints a floating point number with two digits after the point. OctaC adds `%m`, which prints a matrix or a vector.

1.4.1 Sum of Squares

The first program adds up the squares of the numbers from 1 to 5. It shows the basic building blocks of the language: a function, a typed variable, a loop, and output.

```
function main() -> i32 {  
    i32 sum = 0;  
    for (i32 i = 1...5) {  
        sum += i * i;  
    }  
    printf("Sum: %d\n", sum);  
    return 0;  
}
```

The header `function main() -> i32` declares a function. The empty parentheses are its parameter list, and the arrow introduces its return type, so the type of the result is always written after the parameters. Inside the body, `i32 sum = 0;` declares a variable. Every declaration in OctaC follows the same order: the type first, then the name, then the initial value.

The loop `for (i32 i = 1...5)` is a range loop. It declares its own counter, `i`, and counts it from 1 up to and including 5, using the range operator `...`. Each pass adds `i * i` to the running total with the compound assignment `+=`, so the program prints `Sum: 55`.

1.4.2 Matrix Multiplication

The second program shows the feature the language is built around, which is that matrices are a native type rather than something a library provides.

```
function main() -> i32 {  
    matrix<i32> A = [[1, 2], [3, 4]];  
    matrix<i32> B = [[5, 6], [7, 8]];  
    matrix<i32> C = A * B;  
    printf("A * B:\n%m\n", C);  
    return 0;  
}
```

The type `matrix<i32>` is a matrix whose elements are `i32`. A matrix literal is written as a list of rows, and each row is itself a list in brackets, so `[[1, 2], [3, 4]]` is a matrix with two rows and two columns. Because `A` and `B` are initialized with literals at the point of declaration, their shapes are known, and the declaration does not need to state them.

The expression `A * B` is true matrix multiplication. Its value is stored in `C`, and is the matrix `[[19, 22], [43, 50]]`. This is different from element wise multiplication, which has an operator of its own. The expression `A .* B` would multiply the entries position by position, and would give the matrix `[[5, 12], [21, 32]]`. Because the shapes of `A` and `B` are known, the compiler can check that they are compatible for multiplication before the program runs. The format specifier `%m` prints the result.

1.4.3 Matrix-Vector Product

The third program combines the two native types by multiplying a 2×3 matrix by a vector of 3 elements (Ax), which produces a vector of 2 elements.

```
function main() -> i32 {
    matrix<f32> A = [
        [1.0, 2.0, 3.0],
        [4.0, 5.0, 6.0]
    ];
    vector<f32> x = [2.0, 1.0, 0.0];

    // Linear transformation Ax -> 2x1 vector
    vector<f32> y = A * x;

    printf("Result y = Ax: %m\n", y);
    return 0;
}
```

Here `vector<f32>` is a vector of `f32` elements, written as a single bracketed list. The matrix literal is spread over several lines, since whitespace is not significant, and the comment marks what the multiplication computes. The shape of the result follows the same rule as for two matrices: it has as many rows as the left operand, and the multiplication is only valid when the number of columns of `A` matches the number of elements of `x`. Here both are 3, so `y` has 2 elements, namely `[4.0, 13.0]`.

1.4.4 Iterative Convergence

The fourth program introduces conditions and the `until` loop. It halves a value until it drops below a threshold, or until a step limit is reached.

```
function main() -> i32 {
    f32 val = 100.0;
    i32 steps = 0;
    until (val < 1.0 or steps is 20) {
        val /= 2.0;
        steps += 1;
    }
    printf("Converged in %d steps.", steps);
    return 0;
}
```

An `until` loop runs as long as its condition is false, and stops as soon as the condition becomes true. It is the reverse of the `while` loop of C: the condition states when to stop rather than when to continue. The condition here combines two tests. The comparison `val < 1.0` keeps its symbolic form, while equality is written with the word `is`, and the two tests are joined

by `or`. The loop therefore ends when the value has fallen below 1, or after 20 passes, whichever comes first. The second test guards against a loop that would otherwise never end.

The body uses the compound assignments `/=` and `+=`. The value is halved seven times before it falls below 1, so the program prints `Converged in 7 steps.`

1.4.5 Input Validation

The fifth program reads a value from the user and keeps asking until it is valid. It shows the `do...until` loop, which always runs at least once.

```
function main() -> i32 {
  f32 val = 0.0;
  do {
    printf("Enter a nonzero val: ");
    val = input();
  } until (val is not 0.0);
  printf("Accepted: %.2f\n", val);
  return 0;
}
```

In a `do...until` loop the body comes first and the condition is checked afterwards, so the prompt is always shown at least once. The function `input()` reads a value from the user. The condition uses `is not`, the word form of inequality, so the loop repeats until the value entered is not zero. The variable is given the initial value `0.0` so that it holds a defined value before the first read.

1.4.6 GPU-Accelerated Matrix Operations

The last program shows two more parts of the type system: precision and statically declared sizes.

```
function main() -> i32 {
  // Statically sized matrix
  matrix<f16, 1024, 1024> A = identity();

  // can auto-infer size
  matrix<f16> B = A * 2.5;

  matrix<f16, 1024, 1024> C = A * B;

  return 0;
}
```

The element type `f16` is a half precision floating point number. In the declaration of `A`, the dimensions are written as part of the type, so this is a matrix of 1024 by 1024 elements. The matrix `B` is declared without dimensions, because they can be inferred from the expression that initializes it: multiplying `A` by the scalar `2.5` scales every element and keeps the shape.

The last line multiplies two large matrices. Operations on matrices of this size are meant to be offloaded to a GPU automatically when one is available, without the program containing any GPU specific code.

1.5 Looking Ahead

The programs above show the surface of OctaC: typed declarations, native matrices and vectors, two families of arithmetic operators, English word conditions, and four kinds of loops. What they leave open is how a program like these becomes something a machine can run. That is the job of the compiler, which works in phases [1].

The first phase, lexical analysis, reads the program as a stream of characters and splits it into tokens such as `for`, `i32`, `1`, and `...`. Chapter 2 defines these tokens precisely and shows how the lexer recognizes them. The later phases give the tokens a structure and a meaning, and they are where the shape checking described in Section 1.1 takes place. They will be added to this book as they are built.

Chapter 2

Lexical Analysis and the Lexer

The lexical analyzer, or lexer, is the first phase of the compiler. It reads the source program as a stream of characters and groups them into tokens, the units that the parser works with [1]. This chapter defines the full token specification of OctaC, the deterministic finite automata that recognize those tokens [1, 5], and the algorithm the lexer uses to drive them. The source code of the lexer is available at: <https://github.com/Octa-C/compiler/tree/main/lexer>

2.1 Token Specification

This section lists the complete token set for the lexical analyzer, in Table 2.1. Every token belongs to a category, and some belong to a subcategory within it: the loop keywords `for`, `do`, `until`, and `continue`, for example, belong to the Keyword category and the Loops subcategory. The Token column gives the token as it is written in the source, or a descriptive name where one token covers several lexemes. The Values column is filled only for those tokens that match more than one lexeme, or a pattern, such as the scalar datatypes or the integer literals, and lists the lexemes or the regular expression they match. The last column gives the generated name of the token, a short capitalized name such as `FOR` or `ASSIGNMENT`, which is the name used to identify the token from here on.

The logical and equality operators are written as English words, so they are listed among the keywords rather than the operators. The two compiler tokens, EOF and Error, do not correspond to any source text and are produced by the lexer itself.

2.2 Deterministic Finite Automata

The lexer recognizes every token of OctaC with a single deterministic finite automaton, driven by a transition table [1, 5]. The automaton only recognizes the shape of a lexeme, such as a word, a number, or an operator. Which token that lexeme becomes is decided afterwards by a lookup, described in Section 2.3. This keeps the automaton small: reserved words do not need a path of their own, because they have the same shape as identifiers. The subsections below describe the

Category	Subcategory	Token	Values	Generated token
Keyword	Loops	for		FOR
Keyword	Loops	do		DO
Keyword	Loops	until		UNTIL
Keyword	Loops	continue		CONTINUE
Keyword	Function	function		FUNCTION
Keyword	Function	return		RETURN
Keyword	Conditional	if		IF
Keyword	Conditional	else		ELSE
Keyword	Conditional	elseif		ELSEIF
Keyword	Conditional	caseof		CASEOF
Keyword	Conditional	case		CASE
Keyword	Conditional	default		DEFAULT
Keyword	Word-based Logic/Rel Ops	and		AND
Keyword	Word-based Logic/Rel Ops	or		OR
Keyword	Word-based Logic/Rel Ops	not		NOT
Keyword	Word-based Logic/Rel Ops	is		IS
Keyword	Word-based Logic/Rel Ops	in		IN
Keyword	Misc	break		BREAK
Datatype		Scalar	i64, i32, i16, f64, f32, f16, bool	SCALAR
Datatype		vector		VECTOR
Datatype		matrix		MATRIX
Datatype		string		STRING
Punctuation		{		LBRACE
Punctuation		}		RBRACE
Punctuation		(		LPAREN
Punctuation		)		RPAREN
Punctuation		[		LBRACKET
Punctuation		]		RBRACKET
Punctuation		;		SEMICOLON
Punctuation		->		ARROW
Punctuation		,		COMMA
Punctuation		...		ELLIPSIS
Punctuation		:		COLON
Operators	Standard	+-	+, -	PLUSMINUS
Operators	Standard	* / %	*, /, %	STAR_SLASH_MOD
Operators	Matrix	'		TRANSPOSE
Operators	Matrix	.*		DOT_STAR
Operators	Matrix	./		DOT_SLASH
Operators	Relational and Comparison Ops	<		LESS
Operators	Relational and Comparison Ops	>		GREATER
Operators	Relational and Comparison Ops	<=		LESS_EQ
Operators	Relational and Comparison Ops	>=		GREATER_EQ
Operators	Assignment	=, +=, -=, *=, /=		ASSIGNMENT
Identifier		identifier	[a-zA-Z][a-zA-Z0-9_]*	IDENTIFIER
Constants		bool literal	true, false	BOOL_LIT
Constants		int literal	[0-9]+	INT
Constants		float literal	[0-9]+\.[0-9]+ [0-9]+(\.[0-9]+)?[eE][+-]?[0-9]+	FLOAT
Constants		string literal	"[^\"\\n]*"	STRING_LIT
Compiler		EOF		COMPILER_EOF
Compiler		Error		COMPILER_ERROR

Table 2.1: The OctaC token set.

part of the automaton that handles each class of tokens, and each has its own diagram. The last subsection merges them into the one automaton that the lexer runs.

2.2.1 Character Classes

The automaton does not look at individual characters. Each input byte is first mapped to one of 17 character classes, and the transition table has one column per class. The letters `e` and `E` have a class of their own because they can begin the exponent of a number, while everywhere else they behave like any other letter. Table 2.2 lists the classes.

LETTER	A-D	F-Z	a-d	f-z
EXP	E	e		
DIGIT	0-9			
—	—			
WS	\t	\v	\f	\r space
NL	\n			
QUOTE	"			
DOT	.			
PLUS	+			
MINUS	-			
STAR	*			
SLASH	/			
LT	<			
GT	>			
EQ	=			
SIMPLE	%	'	(	) , : ; [] { }
OTHER	any other character			

Table 2.2: Character classes of the lexer.

2.2.2 States

The automaton has 19 states, listed in Table 2.3. A state is accepting when it has an action, which says what the lexeme read so far produces. States without an action are only stepping stones towards a longer lexeme.

State	Reached after reading	Action
DEAD	no valid continuation exists (error state)	none
START	nothing yet	none
IDENT	<code>[a-zA-Z][a-zA-Z0-9_]*</code>	keyword lookup, else identifier
INT	<code>[0-9]+</code>	integer literal
INT_DOT	an integer and a dot, such as <code>12.</code>	none
FLOAT	<code>[0-9]+\.[0-9]+</code>	float literal
EXP	a number and <code>e</code> or <code>E</code>	none
EXP_SIGN	a number, <code>e</code> or <code>E</code> , and a sign	none
FLOAT_EXP	a number with a complete exponent	float literal
OP_DONE	an operator or punctuator that cannot be extended, such as <code>{</code> , <code>-></code> , <code><=</code> , <code>...</code> , <code>.*</code> or <code>=</code>	operator lookup
MINUS	<code>-</code> , which may extend to <code>-></code> or <code>-=</code>	operator lookup
OP_EQ	<code><</code> , <code>></code> , <code>+</code> or <code>*</code> , which may extend with <code>=</code>	operator lookup
SLASH	<code>/</code> , which may extend to <code>/=</code> or begin a comment	operator lookup
COMMENT	<code>//</code> and the rest of the line	skip
DOT	<code>.</code> , waiting for <code>*</code> , <code>/</code> or another <code>.</code>	none
DOT_DOT	<code>..</code> , waiting for a third <code>.</code>	none
STR_BODY	an opening quote and the text after it	none
STR_END	a complete string literal	string literal
WS	a run of whitespace	skip

Table 2.3: States of the lexer DFA.

2.2.3 Identifiers, Keywords, Type Names, and Bool Literals

An identifier has the pattern `[a-zA-Z][a-zA-Z0-9_]*`: a letter followed by any number of letters, digits and underscores. The DFA has one accepting state, `IDENT`, which emits `<IDENTIFIER, value>`. Keywords, type names, and the bool literals `true` and `false` all have the same shape, so the DFA reads them as identifiers as well. After the DFA accepts, the lexer looks up the lexeme in Table 2.4. A lexeme that is in the table gets the token given there in place of `<IDENTIFIER, value>`.

An underscore cannot start an identifier, so it has no transition out of the start state and is reported as a lexical error.

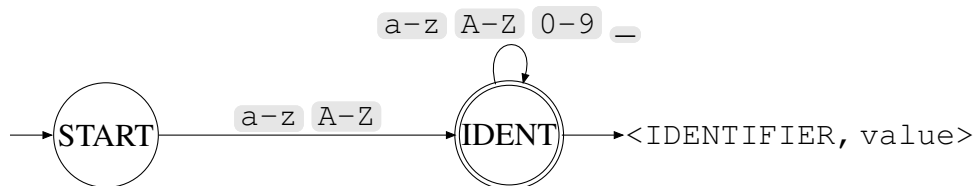


Figure 2.1: DFA for identifiers, keywords, type names, and bool literals.

for	<FOR,>
do	<DO,>
until	<UNTIL,>
continue	<CONTINUE,>
function	<FUNCTION,>
return	<RETURN,>
if	<IF,>
else	<ELSE,>
elseif	<ELSEIF,>
caseof	<CASEOF,>
case	<CASE,>
default	<DEFAULT,>
and	<AND,>
or	<OR,>
not	<NOT,>
is	<IS,>
in	<IN,>
break	<BREAK,>
i64 i32 i16 f64 f32 f16 bool	<SCALAR, value>
vector	<VECTOR,>
matrix	<MATRIX,>
string	<STRING,>
true false	<BOOL_LIT, value>

Table 2.4: Keywords, type names, and bool literals, and the tokens the lookup after IDENT gives them.

2.2.4 Punctuation

A punctuation token is one of the characters `{ } ( ) [ ] ; , : -> .` or `...`. A single character goes from `START` straight to `OP_DONE`. `->` goes through `MINUS`, and `...` goes through `DOT` and `DOT_DOT`. These three states are not accepting in this automaton, because `-` and `.` on their own are not punctuation. `OP_DONE` is accepting, and Table 2.5 gives the token for each lexeme.

{	<LBRACE,>
}	<RBRACE,>
(	<LPAREN,>
)	<RPAREN,>
[	<LBRACKET,>
]	<RBRACKET,>
;	<SEMICOLON,>
->	<ARROW,>
,	<COMMA,>
...	<ELLIPSIS,>
:	<COLON,>

Table 2.5: Punctuation lexemes and the tokens they give.

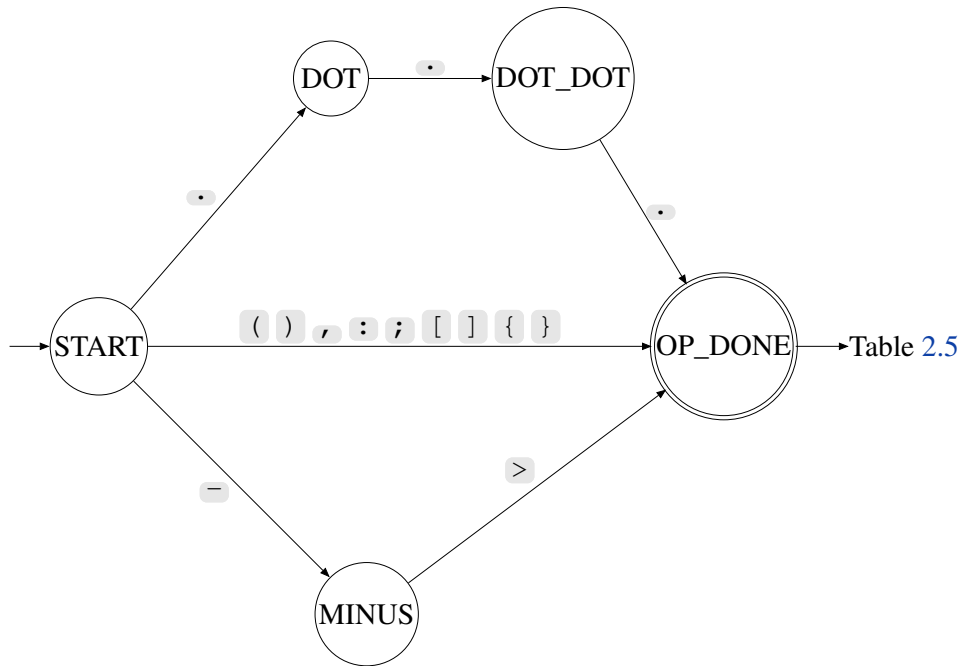


Figure 2.2: DFA for punctuation.

2.2.5 Operators

The operators are `+` `-` `*` `/` `%` `'`, the matrix operators `.*` and `./`, the comparisons `<` `>` `<=` `>=`, and the assignments `=` `+=` `-=` `*=` `/=`. `%` `'` `=` go from `START` straight to `OP_DONE`. `+` `*` `<` `>` go to `OP_EQ`, `-` to `MINUS` and `/` to `SLASH`. These three states are accepting, because each of those characters is an operator on its own, and they continue to `OP_DONE` on `=`. `DOT` is not accepting, because `.` alone is not an operator. It continues to `OP_DONE` on `*` or `/`. The state a lexeme ends in does not fix its token, so Table 2.6 gives the token for each lexeme.

There is no symbolic operator for equality or inequality, since these are written with the words `is` and `is not`.

<code>+</code> <code>-</code>	<code><PLUSMINUS, value></code>
<code>*</code> <code>/</code> <code>%</code>	<code><STAR_SLASH_MOD, value></code>
<code>'</code>	<code><TRANPOSE, ></code>
<code>.*</code>	<code><DOT_STAR, ></code>
<code>./</code>	<code><DOT_SLASH, ></code>
<code><</code>	<code><LESS, ></code>
<code>></code>	<code><GREATER, ></code>
<code><=</code>	<code><LESS_EQ, ></code>
<code>>=</code>	<code><GREATER_EQ, ></code>
<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code>	<code><ASSIGNMENT, value></code>

Table 2.6: Operator lexemes and the tokens they give.

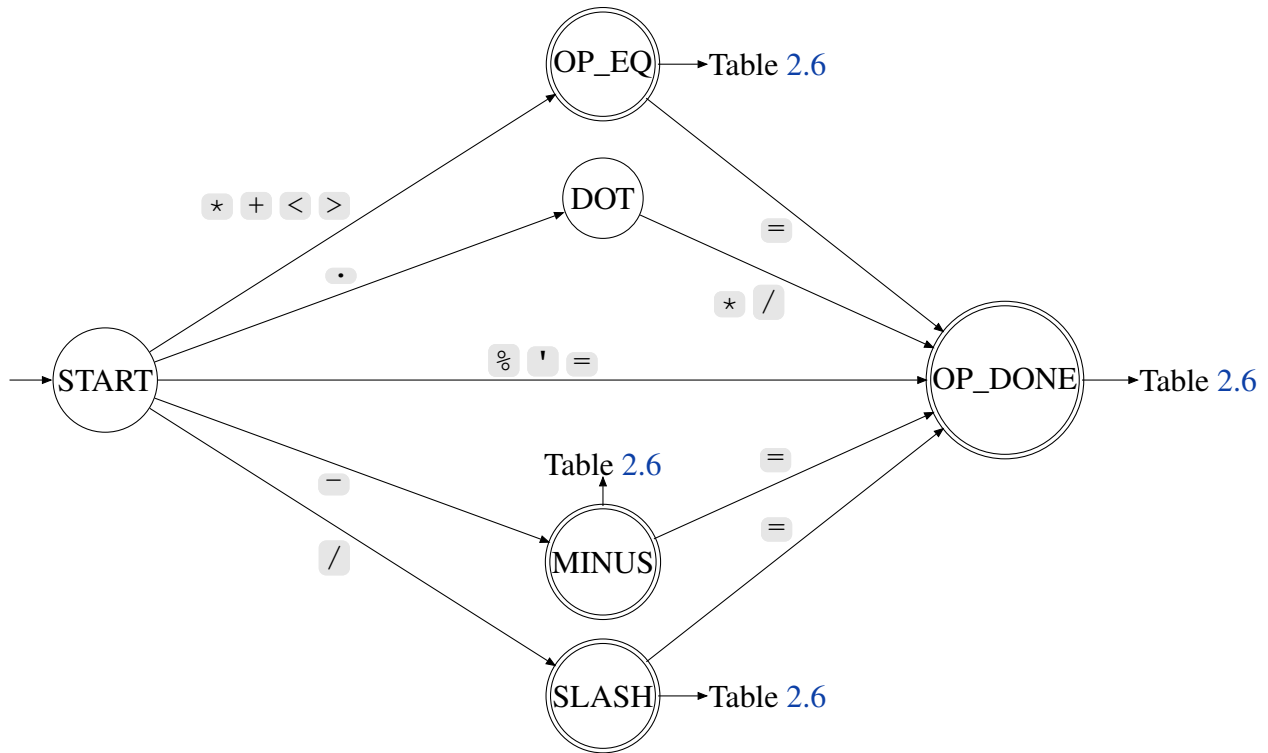


Figure 2.3: DFA for operators.

2.2.6 Constants

A constant is an int literal, a float literal, or a string literal. The bool literals `true` and `false` are words, resolved through the keyword lookup of Section 2.2.3 rather than through this DFA.

- int literal: `[0-9]+`
- float literal: `[0-9]+\.[0-9]+` or `[0-9]+(\.[0-9]+)?[eE][+-]?[0-9]+`
- string literal: `"[^"\n]*"`

Digits are read in `INT`. A dot followed by digits leads through `INT_DOT` to `FLOAT`. An `e` or `E` after `INT` or `FLOAT` leads to `EXP`, an optional sign to `EXP_SIGN`, and digits to `FLOAT_EXP`. A quote starts `STR_BODY`, which loops on every character except a newline or a quote, and the closing quote reaches `STR_END`. `INT`, `FLOAT`, `FLOAT_EXP` and `STR_END` are accepting.

`INT_DOT`, `EXP`, and `EXP_SIGN` are not accepting, so a partial literal such as `1.` or `1e+` is not a number, and the lexer falls back to the longest number that was complete, as described in Section 2.3. A float must have digits on both sides of the dot, so `1.` and `.5` are lexical errors. A digit followed by a letter other than `e` or `E` has no transition, so `12abc` is read as the integer `12` followed by the identifier `abc`. There are no escape sequences in strings, and a string cannot

span lines: a newline inside a string has no transition, so a string that is missing its closing quote never reaches an accepting state and is reported as an error.

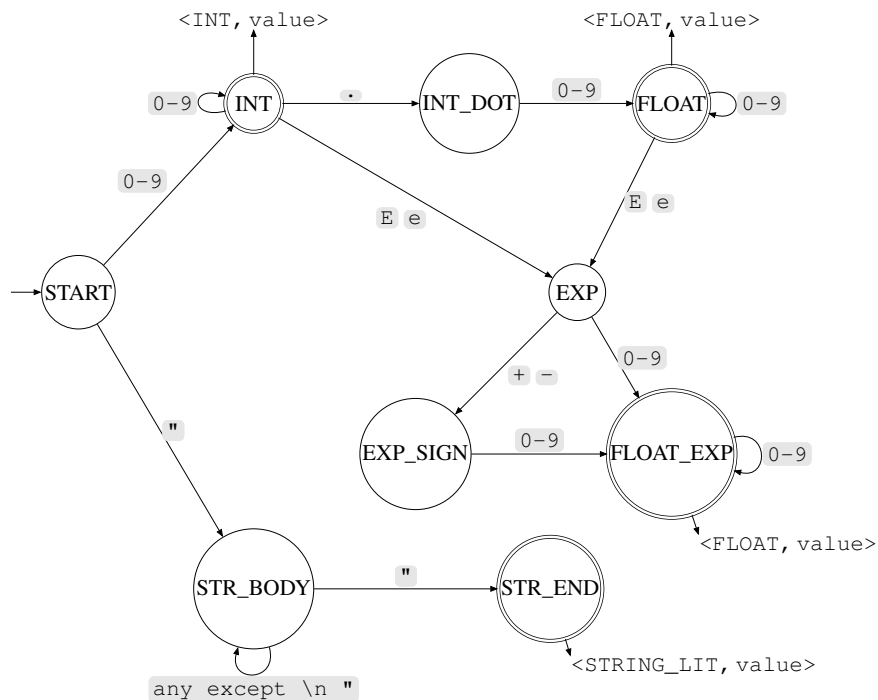


Figure 2.4: DFA for constants.

2.2.7 Compiler Tokens

The end of the input emits `<COMPILER_EOF, >`. The scanner checks for the end of the input before it runs the DFA, so the code has no state for it, and the transition goes straight from the start state to the token. When the next character has no transition and no accepting state was passed, the DFA is in `DEAD`, the error state of the code, and the text read so far is emitted as `<COMPILER_ERROR, >`.

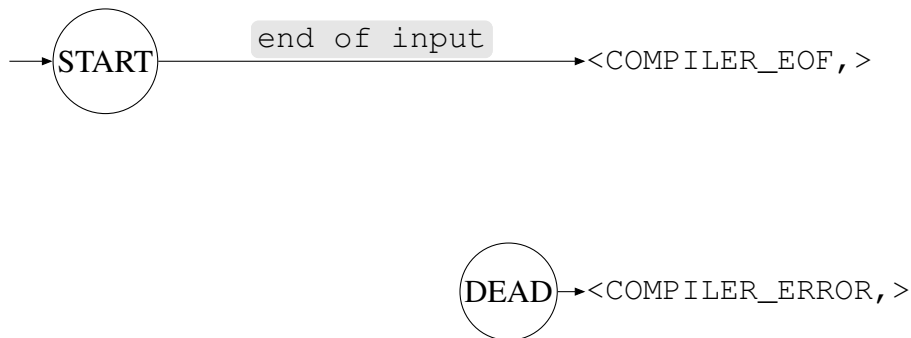


Figure 2.5: DFA for the compiler tokens.

2.2.8 Skipped Input

Skipped input is whitespace and comments. Whitespace is a run of the `WS` and `NL` classes and stays in `WS`. A comment starts with `//` and runs up to, but not including, the next newline, staying in `COMMENT`. `WS` and `COMMENT` are accepting but emit no token. `SLASH` is not accepting in this automaton, because a single `/` is not skipped input: it is the division operator of Section 2.2.5.

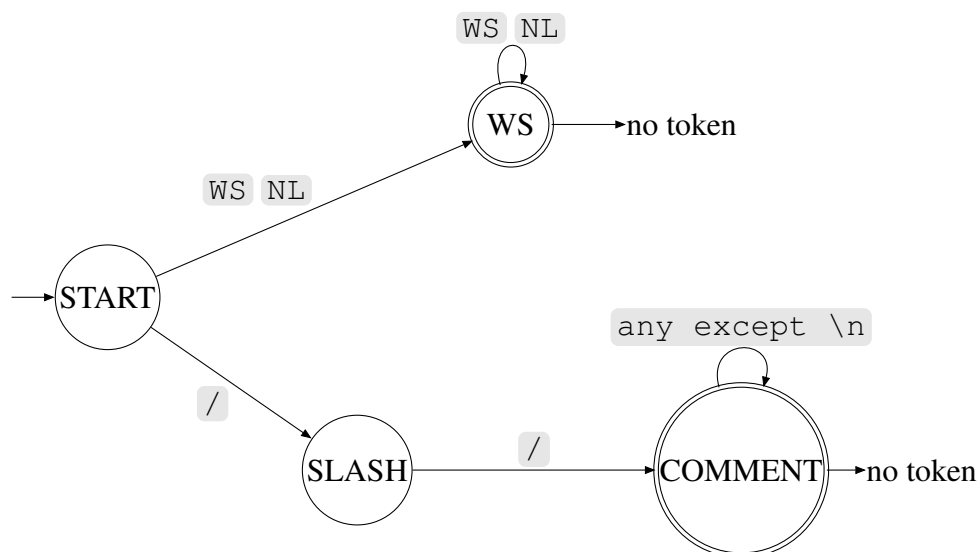


Figure 2.6: DFA for whitespace and comments.

2.2.9 The Merged DFA

The classes are merged into one DFA, the one the lexer runs. Its states carry the names the source gives them.

1. A new start state has a transition for the first character of every class.
2. Classes that share a prefix share states. Keywords, type names and identifiers are all read in `IDENT`, and operators and punctuation end in `OP_DONE`, `MINUS`, `OP_EQ`, `SLASH`, `DOT` and `DOT_DOT`.
3. A state that is accepting for one class stays accepting in the merged DFA, so `MINUS` and `SLASH` are accepting here.
4. `IDENT` emits `<IDENTIFIER, value>`, which Table 2.4 replaces when the lexeme is a keyword or type name. `OP_EQ` and `OP_DONE` emit different tokens depending on the lexeme. The arrow from each of them names the tables that list them.
5. The DFA runs until the next character has no transition. The token of the last accepting state passed is emitted, and if none was passed the consumed text is a `<COMPILER_ERROR, >` token. `DEAD` is not drawn: every missing transition leads to it.

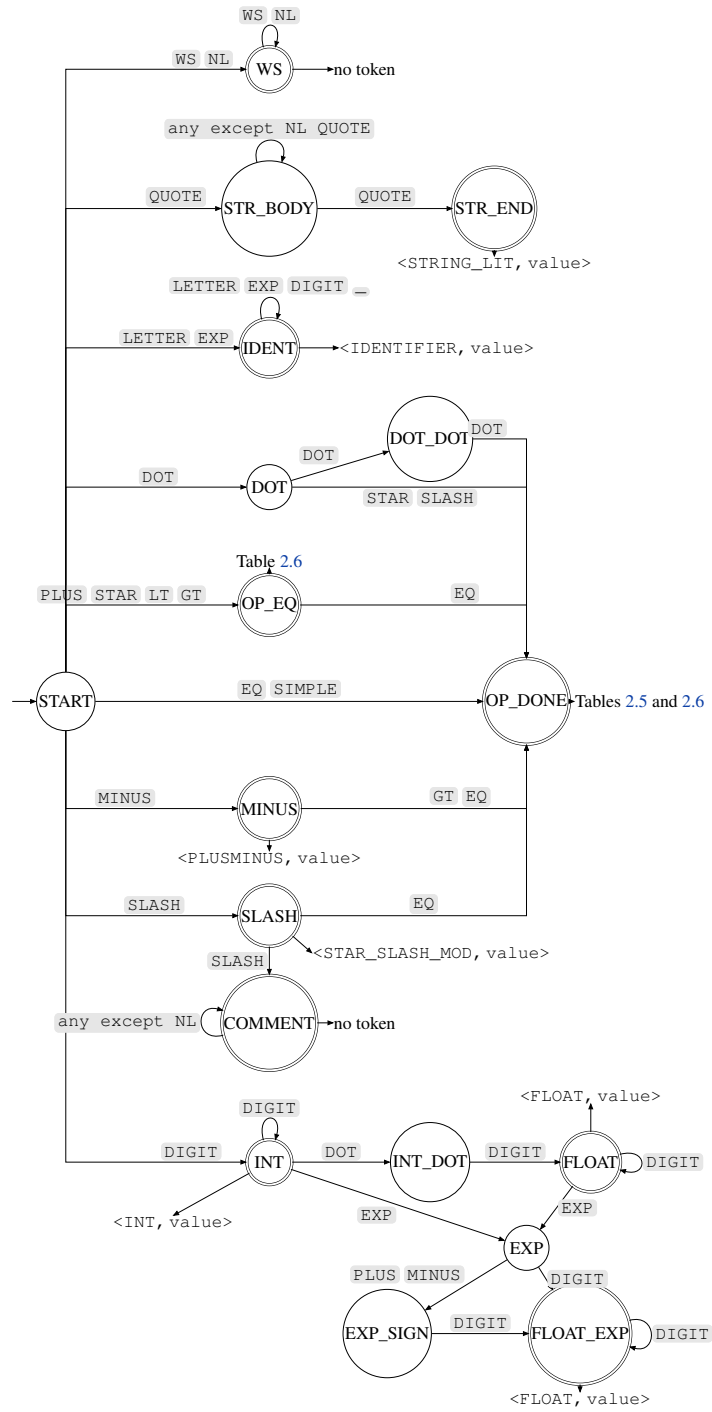


Figure 2.7: The merged DFA of the lexer.

2.3 The Lexical Analyzer Algorithm

The scanner exposes a single operation, `next`, which returns the next token each time it is called. It keeps the source text, the current position, and the current line and column, both counted from 1. The reference implementation is a C++ program.

2.3.1 Maximal Munch

Each call to `next` starts at the current position and walks the transition table for as long as a transition exists. Whenever it enters an accepting state it records that state and the position reached. When it reaches the dead state, or the end of the input, it rewinds to the last recorded position. This is the longest match rule: the lexer always takes the longest prefix that forms a token [1]. The rewinding is what allows one automaton to handle lexemes that share a prefix.

The text `0...2` shows this. The lexer reads `0`, which is accepting, and then a dot, which moves to `INT_DOT`. The next dot has no transition from `INT_DOT`, so the lexer rewinds to the `0` and emits it as an integer literal. Scanning resumes at the first dot, which reaches `OP_DOT` after three dots and emits the ellipsis, and then `2` is scanned on its own. In the same way, `1e+;` is scanned as the integer `1`, the identifier `e`, and the operator `+`, because `EXP` and `EXP_SIGN` are not accepting.

2.3.2 The Scanning Loop

Listing 2.1 shows the body of `next`. The character class mapping is written `classOf`, and `advance` moves the position forward while updating the line and column. The two lookup tables described earlier are used through two functions. The function `lookupKeyword` handles words, and `lookupOperator` handles operators.

Each token also records the line and column where it started, which are taken before the position moves.

2.3.3 Resolving the Token

The action of the last accepting state says how to name the lexeme. A word is looked up in the keyword table, which also holds the type names and the bool literals, and falls back to an identifier. An operator or punctuator is looked up in the operator table. Integer literals, float literals, and string literals are named directly by their state. Whitespace and comments are skipped, and the loop starts again from the new position, so `next` never returns them.

2.3.4 Errors

If the walk never reached an accepting state, the text it consumed becomes the lexeme of an error token, and at least one character is consumed so that scanning always makes progress. The lexer then reports the lexeme on the standard error stream, together with its line and column, the source

```

function next():
  loop:
    if pos >= length(src):
      return Token(COMPILER_EOF)

    start <- pos
    state <- START
    lastAccept <- DEAD
    lastEnd <- start
    cur <- start

    while cur < length(src):
      nxt <- TRANSITION[state][classOf(src[cur])]
      if nxt is DEAD: break
      state <- nxt
      cur <- cur + 1
      if ACTION[state] is not NONE:
        lastAccept <- state
        lastEnd <- cur

    if lastAccept is DEAD:
      end <- start + 1 if cur = start else cur
      lexeme <- src[start .. end)
      advance(end - start)
      return Token(COMPILER_ERROR, lexeme)

    lexeme <- src[start .. lastEnd)
    advance(lastEnd - start)

    switch ACTION[lastAccept]:
      WORD:    type <- lookupKeyword(lexeme)
      INT:     type <- INT
      FLOAT:   type <- FLOAT
      OP:      type <- lookupOperator(lexeme)
      STRING:  type <- STRING_LIT
      SKIP:    continue      // scan again
    return Token(type, lexeme)

```

Listing 2.1: The scanning loop of the lexer.

Input	Reported lexeme
<code>_hidden</code>	<code>-</code>
<code>1.</code>	<code>.</code>
<code>.5</code>	<code>.</code>
<code>1.5.2</code>	<code>.</code>
<code>3..4</code>	<code>..</code>
<code>"unterminated</code>	<code>"unterminated</code>
<code>!=</code>	<code>!</code>
<code>a @ b</code>	<code>@</code>

Table 2.7: Examples of lexical errors.

line it came from, and a marker under the lexeme, and scanning resumes right after it. One run therefore reports every lexical error in a file. Table 2.7 shows how some invalid inputs are reported.

2.3.5 Token Output

The lexer writes one token per line in the form `<TYPE, value>`, ending with the end of file token. Only some tokens have a value part: identifiers, scalar type names, bool, integer, float, and string literals, the arithmetic operators, and the assignment operators. These are the tokens whose Values column is filled in Table 2.1. Every other token is written with an empty value part, for example `<LBRACE, >`.

2.3.6 Sample Runs

To see the algorithm at work, we run the lexer on three statements. For each lexeme, the tables list the states the automaton visits, starting from `START`, followed by the lookup that names the lexeme when there is one, and ending in the token that comes out. The whitespace between lexemes takes the automaton through `WS`, which is skipped, so it does not appear in the tables. After the last lexeme of the input, the lexer returns the end of file token, `<COMPILER_EOF, >`, which is also left out.

Statement 1. The first statement declares a variable. Every lexeme is either a word, a number, or a single character.

```
i32 count = 10;
```

Lexeme	States visited, lookup, and token
i32	START → IDENT → IDENT → IDENT → lookup(keyword table) → <SCALAR, i32>
count	START → IDENT → IDENT → IDENT → IDENT → IDENT → IDENT → lookup(keyword table) → <IDENTIFIER, count>
=	START → OP_DONE → lookup(operator table) → <ASSIGNMENT, =>
10	START → INT → INT → <INT, 10>
;	START → OP_DONE → lookup(operator table) → <SEMICOLON, >

Statement 2. The second statement has a float with an exponent, and a matrix operator that is written without spaces around it.

```
f32 y = 2.5e-3 + a.*b;
```

Lexeme	States visited, lookup, and token
f32	START → IDENT → IDENT → IDENT → lookup(keyword table) → <SCALAR, f32>
y	START → IDENT → lookup(keyword table) → <IDENTIFIER, y>
=	START → OP_DONE → lookup(operator table) → <ASSIGNMENT, =>
2.5e-3	START → INT → INT_DOT → FLOAT → EXP → EXP_SIGN → FLOAT_EXP → <FLOAT, 2.5e-3>
+	START → OP_EQ → lookup(operator table) → <PLUSMINUS, +>
a	START → IDENT → lookup(keyword table) → <IDENTIFIER, a>
.*	START → DOT → OP_DONE → lookup(operator table) → <DOT_STAR, >
b	START → IDENT → lookup(keyword table) → <IDENTIFIER, b>
;	START → OP_DONE → lookup(operator table) → <SEMICOLON, >

Statement 3. The third statement is the range loop, and the range `1...5` is the case where the lexer has to rewind.

```
for (i32 i = 1...5) {
```

Lexeme	States visited, lookup, and token
for	START → IDENT → IDENT → IDENT → lookup(keyword table) → <FOR, >
(	START → OP_DONE → lookup(operator table) → <LPAREN, >
i32	START → IDENT → IDENT → IDENT → lookup(keyword table) → <SCALAR, i32>
i	START → IDENT → lookup(keyword table) → <IDENTIFIER, i>
=	START → OP_DONE → lookup(operator table) → <ASSIGNMENT, =>
1	START → INT → INT_DOT (<i>no move on the next dot, rewind to INT</i>) → <INT, 1>
...	START → DOT → DOT_DOT → OP_DONE → lookup(operator table) → <ELLIPSIS, >
5	START → INT → <INT, 5>
)	START → OP_DONE → lookup(operator table) → <RPAREN, >
{	START → OP_DONE → lookup(operator table) → <LBRACE, >

In the first two runs, every lexeme is accepted at the last state it reaches, and the lexer never has to go back. The word `i32` ends in `IDENT`, and the keyword table turns it into a type token. The float `2.5e-3` is the longest path in the automaton, and it passes through three states that are not accepting. In the third run, the `1` is followed by a dot, which moves the automaton to `INT_DOT`. The next character is another dot, which has no transition there, so the lexer goes back to the last accepting state, which was reached after the `1`, and returns the integer. The scan for the next token then starts at the first dot, and reads all three of them.

2.4 Lexers in the Real World

The lexer of OctaC is deliberately small. This section looks at how lexers are built in practice, what they handle that ours does not, and which simplifications we made.

2.4.1 Lexer Generators

Many lexers are produced by a generator such as Lex [1] or its successor Flex [8]. The programmer writes a list of regular expressions, each with an action. The generator converts them into a deterministic finite automaton and emits a table-driven scanner [1]. Two rules resolve conflicts between patterns: the longest match wins, and among patterns that match the same length, the one listed first wins [1, 8]. Our scanner follows the longest match rule. The second rule is not needed, because keywords are not patterns of their own.

Flex recommends matching identifiers with one pattern and looking keywords up in a table, instead of writing a pattern for each keyword [8], which is what OctaC does. Flex also warns that a scanner which has to back up over several characters is slower, and offers ways to find and remove such cases [8]. Our scanner backs up too, as the run of `1...5` shows, but only over a few

characters. Generators also compress their tables [1]. Ours has 19 states and 17 character classes, which is small enough to be stored as it is.

2.4.2 Hand-Written Lexers

Many production compilers do not use a generator. The lexer of the GNU C preprocessor is coded by hand and is not implemented as a state machine [4], and Clang has a hand-written lexer that scans the source buffer directly [12]. A hand-written lexer gives full control over the speed of the scanner, the quality of error messages, and the handling of features that regular expressions describe poorly. The lexer of OctaC sits between the two approaches. The table is written by hand, but the scanning loop is the same one that a generator would emit.

2.4.3 What Real Lexers Handle

Lexers for established languages deal with many things that OctaC leaves out:

- **Richer literals.** Numbers may be written in hexadecimal, octal, or binary, with digit separators and type suffixes. Strings have escape sequences, raw forms, and may span lines [14].
- **More kinds of comments.** Block comments exist in many languages, and in Rust they can be nested [14].
- **Unicode.** Identifiers may contain letters from any script, following the rules of Unicode Standard Annex 31 [15, 9].
- **Layout.** Python’s lexer produces indent and dedent tokens from the indentation of each line [9]. The lexer of Go inserts semicolons at the ends of lines by a fixed rule, so programs can leave most of them out [11].
- **Preprocessing.** A C lexer works together with the preprocessor, which handles macros and file inclusion [4].

2.4.4 Simplifications in OctaC

Table 2.8 summarizes the differences. Each of the simplifications keeps the automaton small, and each could be lifted later by adding states and character classes to the table.

Concern	Real-world lexers	OctaC
Construction	Generated from regular expressions, or written by hand [8, 12]	One hand-written transition table
Characters	Unicode text, usually as UTF-8	Bytes; anything outside the listed classes is an error
Identifiers	Unicode letters allowed [15]	ASCII letters, digits, and underscores, starting with a letter
Numbers	Hexadecimal, binary, separators, suffixes [14]	Decimal integers and floats, with an optional exponent
Strings	Escapes, raw strings, multiple lines [14]	None of these: one line, no escapes
Comments	Block comments, sometimes nested [14]	Line comments with <code>//</code> only
Layout	Indentation tokens, semicolon insertion [9, 11]	Whitespace is not significant, and semicolons are explicit
Preprocessing	Macros and file inclusion [4]	None
Keywords	Table lookup after matching an identifier [8]	Same
Errors	Diagnostics with recovery	Error token, a diagnostic, and scanning resumes
Input	Streamed through a buffer	The whole file is read into memory

Table 2.8: Real-world lexers compared with the OctaC lexer.

Chapter 3

Syntax Analysis and the Parser

The parser is the second phase of a compiler. It reads the token stream produced by the lexer and builds a parse tree according to the grammar, checking along the way that the tokens form a program that the grammar allows. This chapter gives OctaC's grammar, in LL(1) form, and describes the recursive descent parser built from it. The parser chooses a production from one token of lookahead, builds the tree as it goes, and recovers when it hits a syntax error. The source code of the parser is available at: <https://github.com/Octa-C/compiler/tree/main/parser>

3.1 Grammar Theory

3.1.1 Left-Recursive Grammars

A grammar is left recursive if it has a nonterminal A with a production of the form $A \rightarrow A\alpha \mid \beta$, where β does not itself start with A . Such a rule can never be used by a recursive descent parser: the function for A would call itself before reading any token, and never terminate [1]. The standard fix moves the recursive part into a new nonterminal A' that follows the non-recursive alternative:

$$A \rightarrow \beta A' \quad A' \rightarrow \alpha A' \mid \varepsilon$$

This produces the same language as the original rule, since A is still β followed by zero or more repetitions of α , just written so that the recursion is no longer on the left [1]. A sum written as a left-recursive grammar, for example,

$$G \rightarrow G + T \mid T$$

rewrites the same way, with β as T and α as $+T$:

$$G \rightarrow TG' \quad G' \rightarrow +TG' \mid \varepsilon$$

A grammar also needs left factoring when two productions of the same nonterminal share a prefix, so that the parser cannot tell them apart from the first token. Two productions of the form $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ can be rewritten by factoring the shared prefix out into a new nonterminal, deferring the choice between β_1 and β_2 until after α has been read [1]:

$$A \rightarrow \alpha A' \quad A' \rightarrow \beta_1 \mid \beta_2$$

A statement grammar with three productions that all start with an identifier,

$$S \rightarrow \text{id} = \text{number} \mid \text{id} () \mid \text{id} []$$

needs exactly this treatment: factoring `id` out of all three gives $S \rightarrow \text{id} S'$, with S' then choosing between the three endings:

$$S \rightarrow \text{id} S' \quad S' \rightarrow = \text{number} \mid () \mid []$$

3.1.2 LL(k) Grammars

Eliminating left recursion and left factoring, as above, is not just a fix for recursive descent specifically: it is what turns a grammar into a grammar of a well known class. A parser that reads its input left to right and builds a leftmost derivation, choosing a production for each nonterminal from the next k tokens of input without ever backtracking, is called an $\text{LL}(k)$ parser, and a grammar it can parse this way is an $\text{LL}(k)$ grammar [1]. The first L is the direction the input is read, the second is the kind of derivation built, and k is how far ahead the parser is allowed to look.

A grammar fails to be $\text{LL}(1)$ when some nonterminal has two productions whose possible starting tokens overlap, since then one token of lookahead is not enough to choose between them [1]. That is precisely the problem left recursion and shared prefixes cause, and precisely what eliminating them fixes. $\text{LL}(1)$, with a single token of lookahead, is the common case, and it is exactly what a recursive descent parser gives for free: each nonterminal's function can look at one token and immediately know which production to take.

3.2 The Grammar

3.2.1 Notation

Nonterminals are written in *italics*. A keyword, or a token with several lexemes, is written in capitals, as its token name: `FUNCTION` for the keyword `function`, `SCALAR` for a type name such as `i32`. A token that is a single fixed symbol, such as `{` or `->`, is written as that symbol. Three tokens are shortened for readability: an identifier is written `ID` rather than `IDENTIFIER`, an assignment is written `=` rather than `ASSIGNMENT`, and the end of input is written `$` rather than `COMPILER_EOF`. ϵ is the empty string, and *Program* is the start symbol.

The grammar that follows is OctaC's grammar in $\text{LL}(1)$ form, rewritten by the two transformations of Section 3.1. A sum, for example, was left recursive: $\text{Sum} \rightarrow \text{Sum PLUSMINUS Product} \mid \text{Product}$ became $\text{Sum} \rightarrow \text{Product Sum}'$, $\text{Sum}' \rightarrow \text{PLUSMINUS Product Sum}' \mid \epsilon$. And *Stmt* needed left factoring: a statement starting with an identifier might be an assignment, an indexed assignment, or a function call, and the parser cannot know which until it has read past the identifier, so factoring that identifier out gives $\text{Stmt} \rightarrow \text{ID Stmt}'$, with Stmt' then branching on whatever token follows. The grammar is grouped the way the parser's source code groups its functions: the program and its functions, types, statements, and expressions.

3.2.2 Program and Functions

A program is one or more functions, each with a parameter list, a return type, and a body. *Params* is empty for a function with no parameters, and *ParamList* is a comma separated list of a type and a name, with no trailing comma.

<i>Program</i>	$\rightarrow \text{FUNCTION ID } (\textit{Params}) \rightarrow \textit{Type} \{ \textit{Stmts} \} \textit{Program}'$
<i>Program'</i>	$\rightarrow \text{FUNCTION ID } (\textit{Params}) \rightarrow \textit{Type} \{ \textit{Stmts} \} \textit{Program}' \mid \varepsilon$
<i>Params</i>	$\rightarrow \textit{ParamList} \mid \varepsilon$
<i>ParamList</i>	$\rightarrow \textit{Type} \text{ ID } \textit{ParamList}'$
<i>ParamList'</i>	$\rightarrow , \textit{Type} \text{ ID } \textit{ParamList}' \mid \varepsilon$

3.2.3 Types

A type is a scalar type, `string`, or a `vector` or `matrix` parameterized by an element type and, optionally, its dimensions. The dimensions are themselves omitted when a declaration's shape can be inferred from its initializer, which is a semantic rule checked after parsing rather than one the grammar enforces.

<i>Type</i>	$\rightarrow \text{SCALAR}$ $\mid \text{STRING}$ $\mid \text{VECTOR} < \text{SCALAR } \textit{Type}'$ $\mid \text{MATRIX} < \text{SCALAR } \textit{Type}''$
<i>Type'</i>	$\rightarrow > \mid , \text{INT} >$
<i>Type''</i>	$\rightarrow > \mid , \text{INT} , \text{INT} >$

3.2.4 Statements

Every kind of statement in Chapter 1 has its own production here. *Stmt'* resolves what follows an identifier: an assignment, an indexed assignment or an indexed compound target, or a function call. *Stmt'''* resolves the two forms of `for`, the range form and the `for...in` form, from the single token that follows the loop variable's name. *CaseVal* restricts a `case` label to a literal, optionally signed for a number, which keeps `caseof` to constant labels rather than arbitrary expressions.

<i>Stmts</i>	$\rightarrow \textit{Stmts}'$
<i>Stmts'</i>	$\rightarrow \textit{Stmt } \textit{Stmts}' \mid \varepsilon$

<i>Stmt</i>	$\rightarrow$ <i>Type</i> <i>ID</i> = <i>Expr</i> ; <i>ID</i> <i>Stmt'</i> IF (<i>Expr</i>) { <i>Stmts</i> } <i>ElseIfs</i> <i>Stmt''</i> FOR (<i>Type</i> <i>ID</i> <i>Stmt'''</i> UNTIL (<i>Expr</i>) { <i>Stmts</i> } DO { <i>Stmts</i> } UNTIL (<i>Expr</i>) ; CASEOF (<i>Expr</i>) { <i>Cases</i> <i>Stmt''''</i> RETURN <i>Expr</i> ; BREAK ; CONTINUE ;
<i>Stmt'</i>	$\rightarrow$ = <i>Expr</i> ; [<i>Expr</i>] <i>Stmt'''''</i> (<i>Args</i>) ;
<i>Stmt''</i>	$\rightarrow$ ELSE { <i>Stmts</i> } ε
<i>Stmt'''</i>	$\rightarrow$ = <i>Expr</i> . . . <i>Expr</i>) { <i>Stmts</i> } IN <i>Expr</i>) { <i>Stmts</i> }
<i>Stmt''''</i>	$\rightarrow$ } DEFAULT : <i>Stmts</i> }
<i>Stmt'''''</i>	$\rightarrow$ = <i>Expr</i> ; [<i>Expr</i>] = <i>Expr</i> ;
<i>ElseIfs</i>	$\rightarrow$ <i>ElseIfs'</i>
<i>ElseIfs'</i>	$\rightarrow$ ELSEIF (<i>Expr</i>) { <i>Stmts</i> } <i>ElseIfs'</i> ε
<i>Cases</i>	$\rightarrow$ <i>Cases'</i>
<i>Cases'</i>	$\rightarrow$ CASE <i>CaseVal</i> : <i>Stmts</i> <i>Cases'</i> ε
<i>CaseVal</i>	$\rightarrow$ INT FLOAT STRING_LIT BOOL_LIT PLUSMINUS <i>CaseVal'</i>
<i>CaseVal'</i>	$\rightarrow$ INT FLOAT

3.2.5 Expressions

OctaC resolves operator precedence the usual way a recursive descent grammar does: as a chain of nonterminals, one per precedence level, each built from the next one down. Table 3.4 lists the chain from loosest to tightest. Climbing down it is how the grammar encodes precedence without parentheses: a *Sum* is built from *Products*, so a product never has to yield to a sum that has not finished, and `1 + 2 * 3` is parsed with the multiplication nested inside the addition rather than the other way around.

Table 3.4: Precedence levels of OctaC’s operators.

Level	Nonterminal	Operators
1 (loosest)	<i>Expr</i>	or
2	<i>AndExpr</i>	and
3	<i>NotExpr</i>	not
4	<i>Comp</i>	< > <= >= is is not
5	<i>Sum</i>	+ -

Level	Nonterminal	Operators
6	<i>Product</i>	$\ast / \% \cdot \ast \cdot /$
7	<i>Operand</i>	unary $+$ $-$
8	<i>IndexedData</i>	' (transpose), $[Expr]$ (indexing)
9 (tightest)	<i>Data</i>	literals, identifiers, calls, $(Expr)$, $[Args]$

Comp'' exists because `is not` is two tokens where every other comparison is one: after `is`, a NOT token means `is not` and anything else in $FIRST(Sum)$ means a plain `is`, which is exactly the one token of lookahead the grammar is allowed.

<i>Expr</i>	$\rightarrow AndExpr\ Expr'$
<i>Expr'</i>	$\rightarrow OR\ AndExpr\ Expr' \mid \varepsilon$
<i>AndExpr</i>	$\rightarrow NotExpr\ AndExpr'$
<i>AndExpr'</i>	$\rightarrow AND\ NotExpr\ AndExpr' \mid \varepsilon$
<i>NotExpr</i>	$\rightarrow NOT\ NotExpr \mid Comp$
<i>Comp</i>	$\rightarrow Sum\ Comp'$
<i>Comp'</i>	$\rightarrow < Sum \mid > Sum \mid \leq Sum \mid \geq Sum \mid IS\ Comp'' \mid \varepsilon$
<i>Comp''</i>	$\rightarrow Sum \mid NOT\ Sum$
<i>Sum</i>	$\rightarrow Product\ Sum'$
<i>Sum'</i>	$\rightarrow PLUSMINUS\ Product\ Sum' \mid \varepsilon$
<i>Product</i>	$\rightarrow Operand\ Product'$
<i>Product'</i>	$\rightarrow STAR_SLASH_MOD\ Operand\ Product'$ $\mid \cdot \ast\ Operand\ Product'$ $\mid \cdot /\ Operand\ Product'$ $\mid \varepsilon$
<i>Operand</i>	$\rightarrow PLUSMINUS\ Operand \mid IndexedData$
<i>IndexedData</i>	$\rightarrow Data\ IndexedData'$
<i>IndexedData'</i>	$\rightarrow '\ IndexedData' \mid [Expr]\ IndexedData' \mid \varepsilon$
<i>Data</i>	$\rightarrow ID\ Data'$ $\mid INT$ $\mid FLOAT$ $\mid STRING_LIT$ $\mid BOOL_LIT$ $\mid (Expr)$ $\mid [Args]$
<i>Data'</i>	$\rightarrow (Args) \mid \varepsilon$
<i>Args</i>	$\rightarrow ArgList \mid \varepsilon$
<i>ArgList</i>	$\rightarrow Expr\ ArgList'$

$ArgList' \rightarrow , Expr ArgList' \mid \varepsilon$

3.3 How the Parser Works

The parser is a recursive descent parser over the grammar above, built around a single invariant: because the grammar is LL(1), the next token always picks exactly one production, so no nonterminal's function ever has to guess or back up [1].

3.3.1 One Function per Nonterminal

Every nonterminal in the grammar has its own function. Each function looks at the next token, chooses the one production it matches, and calls the functions for the nonterminals on that production's right-hand side in order, building a tree node as it goes. An empty production is taken whenever the next token does not start the non-empty one, which is always decidable from one token because the grammar is LL(1).

```
function data():
  node <- new node(Data)
  switch next.type:
    case ID:
      node.children += consume()
      node.children += dataP()
    case INT:
    case FLOAT:
    case STRING_LIT:
    case BOOL_LIT:
      node.children += consume()
    case LPAREN:
      node.children += consume()
      node.children += expr()
      node.children += match(RPAREN)
    case LBRACKET:
      node.children += consume()
      node.children += args()
      node.children += match(RBRACKET)
    default:
      reportExpected("an expression")
      throw SyntaxError
  return node
```

Listing 3.1: The function behind *Data*.

Listing 3.1 shows this for *Data*: the `switch` on the next token's type is exactly the one token of lookahead the grammar's LL(1) property guarantees is enough to pick a case. Read against

Section 2.3's scanning loop, the shape is the same idea one level up: where the lexer's loop walks a table one character at a time, the parser's recursion walks the grammar one symbol at a time.

3.3.2 Matching Terminals

A terminal is matched with `match(type)`, which consumes the next token if it has the required type and reports a syntax error otherwise, or `consume()`, which consumes a token already known to be there, such as inside a `switch` on the next token's type. Both add the consumed token to the current node as a leaf.

3.3.3 Building the Tree

A node is either a nonterminal, with one child per symbol of the production that derived it, or a leaf holding the token that matched a terminal. A nonterminal with no children derived the empty string, and is written `(empty)` when the tree is printed. The tree is printed one node per line, indented two spaces per level: a nonterminal as its name from the grammar, with each prime written `P` and the count of primes from two upward (*Stmt'* is `StmtP`, *Stmt''* is `StmtP2`), and a leaf as its token, in the same `<TYPE, value>` form the lexer uses. Where the lexer rewinds to the last accepting state on a dead end, the parser recovers to the next point the grammar can resume from, described next.

3.4 Errors and Recovery

A syntax error is reported the same way a lexical error is: to the standard error stream, with the line and column, the source line, and a marker under the offending text. The message names what the grammar allowed at that point and what was found instead:

```
main.oc:2:13: error: expected an expression but found ';'
  2 |         i32 x = ;
    |                   ^
```

3.4.1 Recovery

Reporting an error throws, which unwinds to the nearest recovery point rather than to the top of the parse. Two recovery points exist, matching the two places a broken construct can be skipped without losing track of where the rest of the program begins.

- **Inside a block.** A broken statement is skipped up to the next `;` at the same brace depth it started at, or to the `}` that closes a block the statement itself opened, whichever comes first. A count of unmatched `{` against `}` is kept throughout, so skipping never mistakes a brace that belongs to a nested block for the one that closes the enclosing one.

- **Between functions.** A broken function, or a broken top-level construct, is skipped up to the next `FUNCTION` keyword, or to the end of input.

Because recovery always lands on a token that can start a new statement, a new function, `case`, or `default`, parsing carries on from there rather than stopping at the first error. One run can therefore report many syntax errors in a file, the same way one run of the lexer reports many lexical errors. Two errors reported at the same token are collapsed into one, since a single bad token otherwise tends to be rejected by more than one function on the way back up. No tree is produced for a file that had any syntax error, even though the recovered parse continues to the end of the input to keep finding them. Table 3.6 shows some invalid inputs and the diagnostic each one produces.

Input	Diagnostic
<code>i32 x = ;</code>	expected an expression but found ';'
<code>function -> i32 {</code>	expected an identifier but found '->'
<code>for (i32 i 0...1) {</code>	expected an assignment operator or 'in' but found '0'
<code>matrix<f32, 2> m = 1;</code>	expected ',,' but found '>'

Table 3.6: Examples of syntax errors.

A file that ends with an unclosed `if` shows recovery reaching the end of input rather than a token: since no `}` ever arrives to skip to, the error is reported once, at the end of input, rather than repeated for every nesting level left open.

```
if (x) {
    y = 1;
(end of input)
```

This produces a single diagnostic, `4:1: expected '}' but found end of input`.

3.5 Sample Parses

3.5.1 A Minimal Program

A short, complete OctaC program is a single function that returns immediately:

```
function main() -> i32 {
    return 0;
}
```

Even this short a program produces a deep tree, because every expression, here just the literal `0`, passes through the full precedence chain of Section 3.2.5 on its way down to *Data*. Listing 3.2 is the tree `-emit-parse-tree` prints for it, and Figure 3.1 draws the same tree as a diagram.

```

Program
  <FUNCTION,>
  <IDENTIFIER, main>
  <LPAREN,>
  Params (empty)
  <RPAREN,>
  <ARROW,>
  Type
    <SCALAR, i32>
  <LBRACE,>
  Stmts
    StmtsP
    Stmt
      <RETURN,>
      Expr
        AndExpr
          NotExpr
            Comp
              Sum
                Product
                  Operand
                    IndexedData
                      Data
                        <INT, 0>
                        IndexedDataP (empty)
                        ProductP (empty)
                        SumP (empty)
                        CompP (empty)
                        AndExprP (empty)
                        ExprP (empty)
                      <SEMICOLON,>
                    StmtsP (empty)
                  <RBRACE,>
                ProgramP (empty)

```

Listing 3.2: The parse tree of the program above.

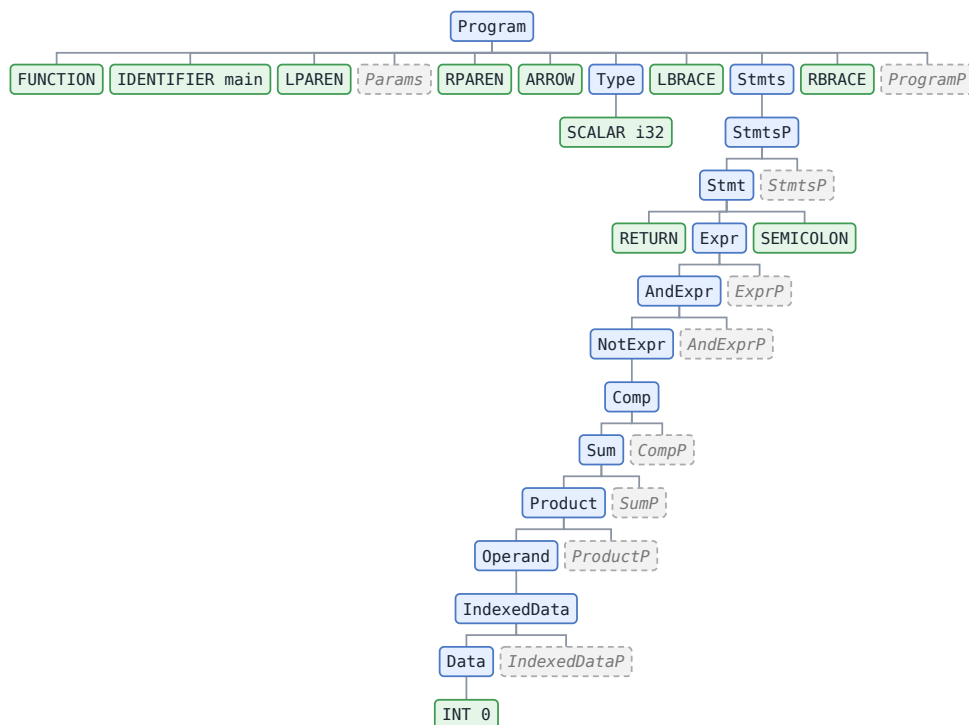


Figure 3.1: The parse tree of the minimal program.

3.5.2 Operator Precedence

The next example shows the precedence chain actually branching, rather than passing straight through:

```
function foo() -> i32 {
  i32 x = 1 + 2 * 3;
}
```

Listing 3.3: A declaration whose initializer branches.

Figure 3.2 draws its tree. The addition is the outer operation: *Sum* holds the 1, and its *SumP* child holds the + together with the *Product* for 2 * 3, which is where the multiplication is parsed, one level further down, before the addition around it is complete. This is precedence shown structurally, with no parentheses needed to force the multiplication to bind tighter.

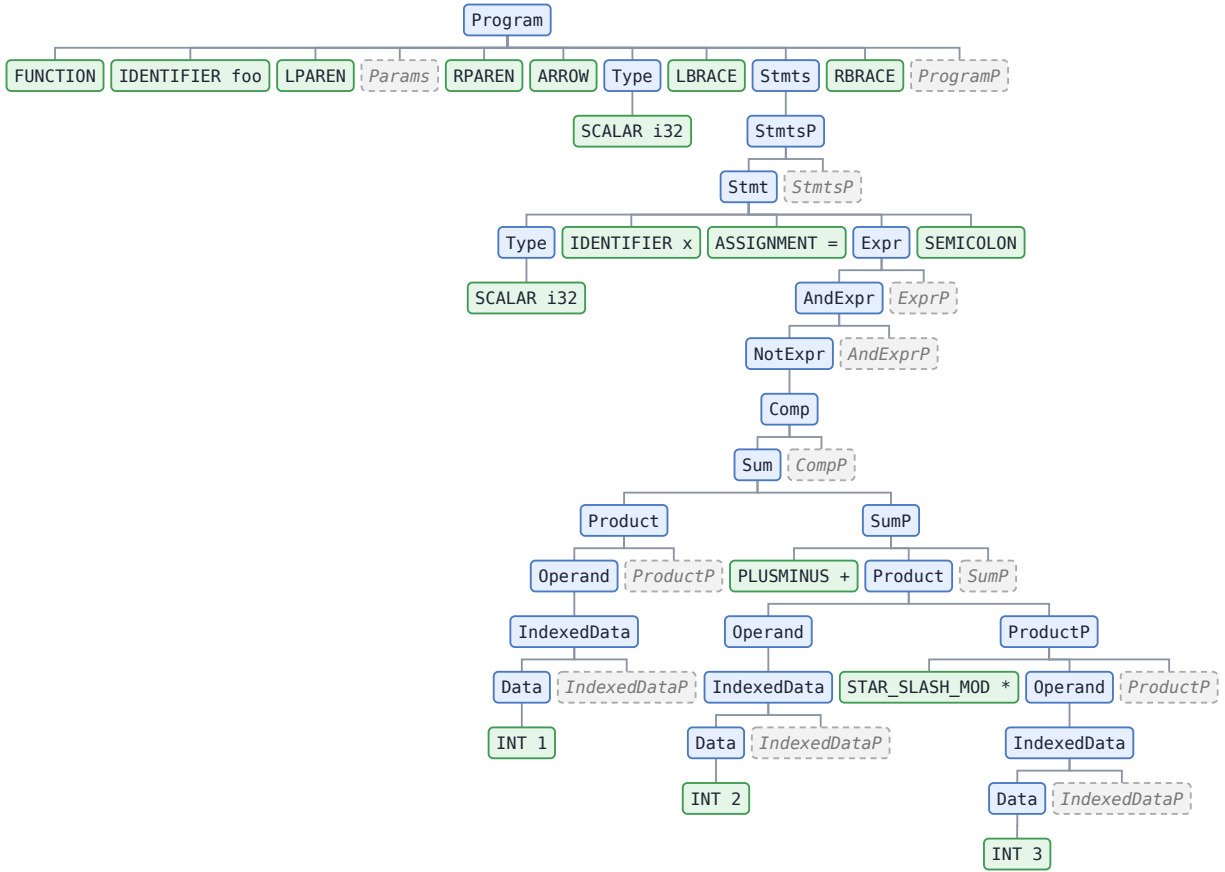


Figure 3.2: The parse tree of Listing 3.3.

In both figures, a nonterminal node is blue, a token is green, and a nonterminal that derived the empty string is grey and dashed. Most of the grey around the edges of Figures 3.1 and 3.2 is exactly this: nearly every $\dots P$ nonterminal in a short program derives the empty string, since the recursion it stands for never runs for more than a round or two.

3.6 Parsers in the Real World

The parser of OctaC, like its lexer, is hand-written rather than generated, and deliberately small. This section looks at how parsers are built in practice, and which simplifications ours makes.

3.6.1 Parser Generators and Parsing Methods

Many parsers are produced by a generator, the same way many lexers are. Bison, upward compatible with the older Yacc, converts a grammar annotated with actions into a parser, using LALR(1) tables by default [3]. LALR is a bottom-up method that accepts a larger class of grammars than LL(1), at the cost of a less direct relationship between the grammar and the code that results [1].

OctaC’s grammar is restricted to LL(1) on purpose, so that the parser can be a direct, hand-written translation of it, with no generated code and no table beyond the one a human reads from the grammar itself.

3.6.2 Hand-Written Recursive Descent

Recursive descent is also a common choice for production compilers, not only a teaching technique. Clang’s parser is hand-written recursive descent over the grammar of C and C++ [12], and CPython’s parser has been a hand-written PEG parser, a close relative of recursive descent with unlimited lookahead and backtracking, since Python 3.9 [16]. A hand-written parser gives the same advantages a hand-written lexer does: direct control over error messages and recovery, and no dependency on a generator as part of the build.

3.6.3 Error Recovery

Real parsers recover from a syntax error so that one run can report more than the first one found, the same reason ours does. The general strategy, skipping input until a token is reached that the grammar allows at some enclosing point, is called panic mode recovery [1]. OctaC’s parser is a direct application of it, with exactly two recovery points chosen for the shape of the grammar: the end of a statement, and the start of a function.

3.6.4 Simplifications in OctaC

Table 3.7 summarizes the differences.

Concern	Real-world parsers	OctaC
Construction	Generated from a grammar, or hand-written [3, 12]	Hand-written recursive descent
Grammar class	LALR or PEG, each more permissive than LL(1) [1, 16]	LL(1), by construction
Lookahead	Unbounded, with backtracking in a PEG parser [16]	Exactly one token, never backtracks
Error recovery	Panic mode, or finer-grained strategies tuned per construct [1]	Panic mode, at two recovery points
Tree	Often folded into an AST during parsing	A literal parse tree, one node per grammar symbol
Ambiguity	Resolved by precedence declarations in the generator	Resolved in the grammar itself, by construction

Table 3.7: Real-world parsers compared with the OctaC parser.

Conclusion

Conclusion will be updated once the project is complete....

Bibliography

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2006.
- [2] John W. Eaton, David Bateman, Søren Hauberg, and Rik Wehbring. GNU octave documentation. <https://docs.octave.org/latest/>.
- [3] Free Software Foundation. Bison: GNU parser generator. <https://www.gnu.org/software/bison/manual/bison.pdf>.
- [4] Free Software Foundation. The GNU C preprocessor internals. <https://gcc.gnu.org/onlinedocs/cppinternals/>.
- [5] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson, 3rd edition, 2006.
- [6] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 2nd edition, 1988.
- [7] Steve Klabnik and Carol Nichols. The rust programming language. <https://doc.rust-lang.org/book/>.
- [8] Vern Paxson, Will Estes, and John Millaway. Lexical analysis with flex. <https://westes.github.io/flex/manual/>.
- [9] Python Software Foundation. The python language reference. <https://docs.python.org/3/reference/>.
- [10] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 4th edition, 2013.
- [11] The Go Authors. The go programming language specification. <https://go.dev/ref/spec>.
- [12] The LLVM Project. “clang” CFE internals manual. <https://clang.llvm.org/docs/InternalsManual.html>.
- [13] The MathWorks, Inc. MATLAB documentation. <https://www.mathworks.com/help/matlab/>.

- [14] The Rust Project Developers. The rust reference. <https://doc.rust-lang.org/reference/>.
- [15] The Unicode Consortium. Unicode standard annex #31: Unicode identifier and pattern syntax. <https://www.unicode.org/reports/tr31/>.
- [16] Guido van Rossum, Pablo Galindo, and Lysandros Nikolaou. PEP 617 – new PEG parser for CPython. <https://peps.python.org/pep-0617/>.
- [17] Zig Software Foundation. Zig language reference. <https://ziglang.org/documentation/>.